1. 트랜잭션 암호화 프로세스

개요

Sapphire에서 트랜잭션의 기밀성을 보장하기 위해 클라이언트에서 TEE까지의 end-to-end 암호화 프로세스를 상세히 설명합니다.

1.1 암호화 아키텍처

전체 흐름도

```
클라이언트 → [암호화] → 네트워크 → [복호화] → TEE → [실행]
→ [암호화된 상태 저장]
```

사용되는 암호화 알고리즘

- **키 교환**: X25519 (Curve25519)
- **대칭 암호화**: DeoxysII (AEAD)
- **해시**: SHA-512/256, SHA-384, SHA-512
- **랜덤 생성**: KMAC256, cSHAKE 기반 VRF

1.2 키 생성 및 교환

X25519 키 쌍 생성

```
function  generateCurve25519KeyPair(bytes memory  pers)

internal  view  returns (Curve25519PublicKey pk,
 Curve25519SecretKey sk)

{

bytes memory scalar = randomBytes(32, pers);

// RFC 7748 §5에 따른 비트 조작

scalar[0] &= 0xf8;  // 작은 부분군 공격 방지를 위해 8의 배수로 만듦

scalar[31] &= 0x7f;  // 2^255 - 19 미만으로 클램핑

scalar[31] |= 0x40;  // 2^254 이상으로 클램핑

(bool success, bytes memory pkBytes) =
CURVE25519_PUBLIC_KEY.staticcall(scalar);

require(success, "gen curve25519 pk: failed");

return (

Curve25519PublicKey.wrap(bytes32(pkBytes)),

Curve25519SecretKey.wrap(bytes32(scalar))

);

}
```

대칭키 유도

```
function  deriveSymmetricKey(

Curve25519PublicKey peerPublicKey,

Curve25519SecretKey secretKey

) internal  view  returns (bytes32) {
```

```
(bool success, bytes memory symmetric) = DERIVE_KEY.staticcall(
abi.encode(peerPublicKey, secretKey)
);
require(success, "deriveSymmetricKey: failed");
return bytes32(symmetric);
}
```

1.3 Calldata 암호화 과정

1단계: 내부 암호화

```
function _encryptInner(
bytes memory in_data,
Sapphire.Curve25519SecretKey in_x25519_secret,
bytes15 nonce,
bytes32 peerPublicKey
) view returns (bytes memory out_encrypted) {
// CBOR 형식으로 데이터 래핑
bytes memory plaintextEnvelope = abi.encodePacked(
hex"a1", // map(1)
hex"64", // text(4) "body"
"body",
CBOR.encodeBytes(in_data)
```

```
);


// DeoxysII로 암호화
out_encrypted = Sapphire.encrypt(
_deriveKey(peerPublicKey, in_x25519_secret),
nonce,
plaintextEnvelope,
""
);
}
```

2단계: 외부 봉투 생성

```
function encryptCallData(
bytes memory in_data,
Sapphire.Curve25519PublicKey myPublic,
Sapphire.Curve25519SecretKey mySecret,
bytes15 nonce,
uint256 epoch,
bytes32 peerPublicKey
) view returns (bytes memory out_encrypted) {
if (in_data.length == 0) {
return "";
```

```
}

bytes memory inner = _encryptInner(in_data, mySecret, nonce, peerPublicKey);

return  abi.encodePacked(
hex"a2", // map(2)
hex"64", // text(4) "body"
"body",
hex"a4", // map(4)
hex"62", // text(2) "pk"
"pk",
hex"5820", // bytes(32)
myPublic,
hex"64", // text(4) "data"
"data",
CBOR.encodeBytes(inner), // bytes(n) inner
hex"65", // text(5) "epoch"
"epoch",
CBOR.encodeUint(epoch), // unsigned(epoch)
hex"65", // text(5) "nonce"
"nonce",
hex"4f", // bytes(15) nonce
nonce,
```

```
hex"66", // text(6) "format"
"format",
hex"01"  // unsigned(1)
);
}
```

1.4 CBOR 인코딩

CBOR 데이터 구조

암호화된 calldata는 다음과 같은 CBOR 구조를 가집니다:

```
{
"body":  {
"pk":  "<32바이트 공개키>",
"data":  "<암호화된 내부 데이터>",
"epoch": <에포크 번호>,
"nonce":  "<15바이트 논스>"
},
"format":  1
}
```

내부 데이터 구조

```
{
"body": "<실제 트랜잭션 데이터>"
}
```

1.5 DeoxysII 암호화

알고리즘 특성

- **AEAD (Authenticated Encryption with Associated Data)**
- **128비트 블록 크기**
- **256비트 키 크기**
- **120비트 논스 (15바이트)**

암호화 함수

```
function encrypt(
bytes32 key,
bytes32 nonce,
bytes memory plaintext,
bytes memory additionalData
) internal view returns (bytes memory) {
(bool success, bytes memory ciphertext) = ENCRYPT.staticcall(
abi.encode(key, nonce, plaintext, additionalData)
);
```

```
require(success, "encrypt: failed");

return ciphertext;

}
```

복호화 함수

```
function decrypt(

bytes32 key,

bytes32 nonce,

bytes memory ciphertext,

bytes memory additionalData

) internal view returns (bytes memory) {

(bool success, bytes memory plaintext) = DECRYPT.staticcall(

abi.encode(key, nonce, ciphertext, additionalData)

);

require(success, "decrypt: failed");

return plaintext;

}
```

1.6 보안 고려사항

논스 관리

- **15바이트 랜덤 논스** 사용

- **재사용 방지**: 각 트랜잭션마다 새로운 논스 생성
- **충돌 확률**: 2^120 공간에서 충돌 확률 극히 낮음

키 관리

- **임시 키 쌍**: 각 트랜잭션마다 새로운 키 쌍 생성
- **키 유도**: X25519 ECDH를 통한 안전한 대칭키 유도
- **키 소거**: 사용 후 메모리에서 키 정보 안전하게 제거

메타데이터 보호

- **에포크 정보**: 리플레이 공격 방지
- **공개키 포함**: 수신자가 올바른 키로 복호화할 수 있도록 보장
- **형식 버전**: 향후 호환성을 위한 버전 정보

1.7 성능 최적화

가스 비용

- **암호화**: 50,000 기본 + 입력 워드당 100 가스
- **복호화**: 50,000 기본 + 입력 워드당 100 가스
- **키 유도**: 100,000 가스

배치 처리

- 여러 트랜잭션을 하나의 배치로 처리하여 암호화 오버헤드 감소
- 최대 배치 크기: 1,000 트랜잭션

1.8 클라이언트 구현 예시

TypeScript 클라이언트

```
import { Sapphire } from '@oasisprotocol/sapphire-paratime';

// 트랜잭션 데이터 암호화
const encryptedCalldata = await Sapphire.encryptCallData(
provider,
{
to: contractAddress,
data: calldata,
value: ethers.utils.parseEther("1.0")
}
);

// 암호화된 트랜잭션 전송
const tx = await wallet.sendTransaction(encryptedCalldata);
```

Go 클라이언트

```
import "github.com/oasisprotocol/sapphire-paratime/clients/go"

// 트랜잭션 암호화
encryptedTx, err := sapphire.EncryptTransaction(ctx, tx)
```

```
if err != nil {
return err
}

// 전송
receipt, err := client.SendTransaction(ctx, encryptedTx)
```

결론

Sapphire의 트랜잭션 암호화 프로세스는 X25519 키 교환과 DeoxysII AEAD 암호화를 조합하여 강력한 end-to-end 기밀성을 제공합니다. CBOR 인코딩을 통한 구조화된 메타데이터 관리와 함께, 안전하고 효율적인 기밀 트랜잭션 처리를 가능하게 합니다.